Etapa 4: Diseño de la Arquitectura General del Sistema.

Versión actualizada al 30 de octubre de 2026, con la arquitectura implementada en la Fase 4.

Para lograr un reconocimiento de voz confiable, evitar la saturación de memoria y cómputo del microcontrolador y mantener el sistema funcionando dentro de la casa aun sin internet, el sistema adopta un modelo desacoplado y distribuido de cómputo local (*edge computing*). Se separa la capa de adquisición y actuación en tiempo real (nodo ESP32) de la capa de voz (satélite en la estancia) y de la capa de procesamiento central (servidor local). Todos los componentes se comunican por un broker MQTT local; solo las alertas a familiares y el acceso remoto al panel salen a internet.

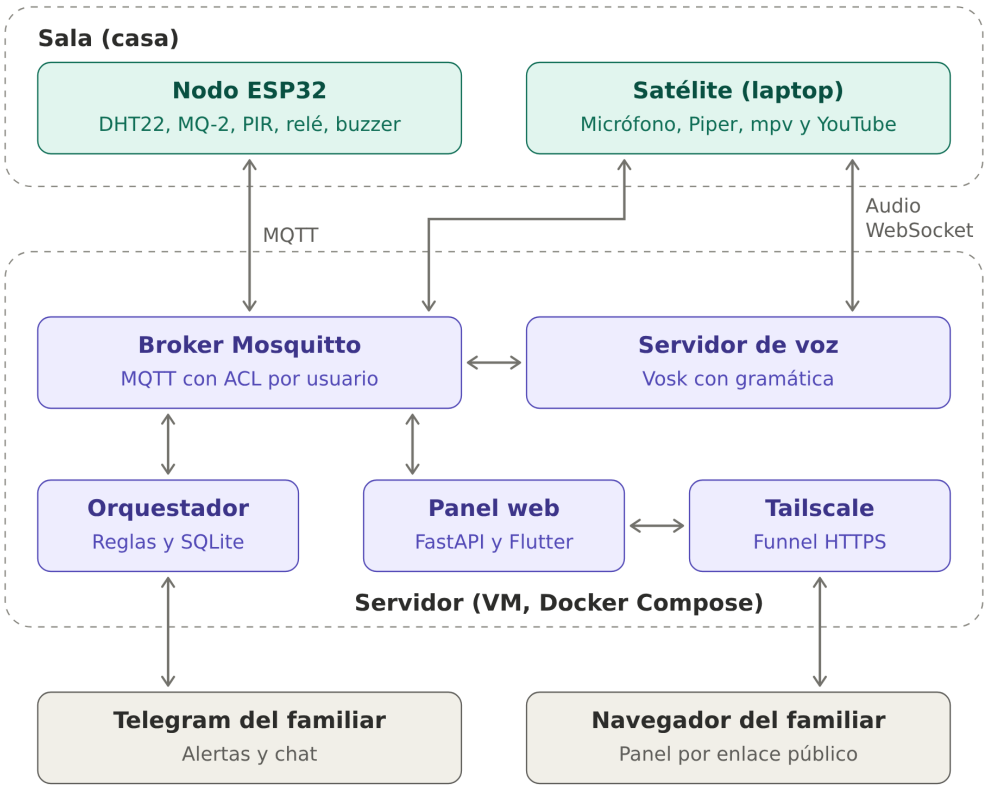


Figura 1. Arquitectura general del sistema implementado.

1. Nivel de Control Físico y Telemetría (Nodo ESP32)

- **Función:** Actuar como cliente perimetral dedicado a la adquisición de variables físicas (presencia, temperatura, humedad y gas) y al accionamiento eléctrico de la luz mediante un relé optoacoplado.
- **Comportamiento:** No procesa audio. Publica lecturas periódicas y eventos (gas, movimiento, cambios de la luz) por MQTT sobre Wi-Fi, con QoS 1 para las alertas, y recibe órdenes de encendido y apagado. Ante gas activa una alarma sonora local (buzzer) que no depende de la red ni del servidor. El botón físico alterna la luz aunque el servidor no esté disponible.

2. Nivel de Voz (Satélite en la estancia y Servidor de voz)

- **Función:** Captar la voz del usuario con un micrófono inalámbrico conectado a una laptop en la estancia (satélite), enviar el audio al servidor de voz y reproducir las respuestas habladas.
- **Activación:** El usuario habla mientras mantiene pulsada una tecla del satélite, o un familiar mantiene pulsado el botón «Mantener para hablar» del panel. La palabra de activación

(*wake-word*) quedó como trabajo futuro.

- **Reconocimiento (STT):** El servidor de voz usa Vosk en modo gramática: reconoce un conjunto cerrado de frases en español, con frases señuelo que absorben lo que no es una orden. Así se reducen las órdenes falsas y la respuesta tarda milisegundos.
- **Respuesta (TTS):** El satélite genera la respuesta con Piper (voz es_MX-claude-high) y también lee en voz alta las alertas, los recordatorios y los mensajes de los familiares. Además puede reproducir videos de YouTube cuando el usuario lo pide.

3. Nivel de Procesamiento Central (Servidor local)

- **Función:** Una máquina virtual con Ubuntu Server en el servidor de la casa ejecuta, con Docker Compose, el broker MQTT, el servidor de voz, el orquestador, el panel web y Tailscale.
- **Orquestador:** Programa en Python (asyncio) que traduce el texto reconocido a intenciones (encender o apagar la luz, ayuda, cancelar, hora, fecha, temperatura, recordatorios, «estoy bien», video), aplica las reglas de gas e inactividad, programa los recordatorios y guarda alertas, conversación y familiares en SQLite.
- **Panel web:** API en FastAPI y aplicación en Flutter web, con estado de la sala en tiempo real, alertas e historial, recordatorios, mensajes, familiares e invitaciones, control de la luz y botón para hablar.

4. Nivel de Comunicación y Red Local

- **Broker MQTT local (Mosquitto, en el servidor):** Canal de mensajes ligero, desacoplado y en tiempo real. Cada componente tiene su propio usuario y una lista de control de acceso (ACL) que limita qué tópicos puede leer o escribir. Garantiza que la automatización y el control por voz sigan funcionando dentro del hogar aun si se pierde la conexión a internet.
- **Audio:** El audio no viaja por MQTT, sino por un WebSocket entre el satélite y el servidor de voz, autenticado con un token.

5. Nivel de Enlace Externo

- **Alertas:** Ante una petición de ayuda, gas o inactividad, el orquestador envía un mensaje por Telegram a todos los familiares registrados, con los botones «Abrir panel» y «Ya lo atendí». Los envíos pasan por una cola persistente que reintenta si no hay internet. Los SMS y llamadas automáticas quedan como trabajo futuro.
- **Acceso remoto:** El panel se publica con HTTPS mediante Tailscale Funnel, sin abrir puertos del router; el broker MQTT nunca queda expuesto a internet.

Etapa 5: Selección de Componentes de Hardware y Software.

Versión actualizada al 30 de octubre de 2026, con los componentes usados en el prototipo.

Categoría	Componente seleccionado	Función técnica en el prototipo	Justificación técnica
Unidad de procesamiento central (servidor)	Máquina virtual con Ubuntu Server y Docker en el servidor de la casa (Proxmox)	Broker MQTT, servidor de voz, orquestador, panel web y Tailscale.	Más potencia que una Raspberry Pi para el reconocimiento de voz; los respaldos de la máquina virtual permiten volver atrás, y Docker permite reinstalar el sistema igual en otra casa. Una Raspberry Pi 5 o mini PC sigue siendo opción para la instalación definitiva.
Satélite de voz	Laptop con Linux en la estancia	Captura del audio, envío al servidor, síntesis de voz con Piper y reproducción de videos.	Equipo disponible, con audio y pantalla integrados; Piper corre en local con buena calidad.
Controlador de E/S y sensores	ESP32 NodeMCU-32S	Interfaz directa con sensores, relé, buzzer y botón; cliente MQTT.	Maneja señales digitales y analógicas en tiempo real a bajo costo y bajo consumo, liberando al servidor de la capa eléctrica.
Captura de voz	Micrófono inalámbrico con receptor USB-C	Captación de la voz del usuario a distancia.	Sustituye al ReSpeaker: se consigue con facilidad, funciona en Linux sin controladores y permite hablar sin acercarse al equipo. El micrófono de solapa de 3.5 mm se descartó por su baja calidad.
Salida de audio / confirmación	Bocina de la laptop o bocina externa conectada a ella	Respuestas habladas, alertas, recordatorios y mensajes leídos en voz alta.	Reproducción directa de las respuestas sintetizadas, con volumen suficiente para personas con dificultades auditivas.
Detección ambiental	Sensor PIR HC-SR501 + sensor DHT22	Movimiento para el aviso de inactividad; temperatura y humedad de la estancia.	Supervisión pasiva continua cableada al ESP32 (GPIO 35 y GPIO 32) sin intervención del usuario.
Detección de riesgo	Sensor de gas y humo MQ-2 + buzzer activo	Detección de fugas de gas o humo y alarma sonora local.	Salida analógica a GPIO 34 por un divisor de voltaje (10 kΩ / 20 kΩ) para no exceder 3.3 V; el buzzer se activa con un transistor 2N2222 y un diodo 1N4148 de protección.
Actuación	Módulo de relés optoacoplado de 2 canales (5 V), con un canal en uso	Encendido y apagado de la luz; en el prototipo, un foco LED USB.	Bobina alimentada a 5 V (JD-VCC) y lógica a 3.3 V para aislar al ESP32. En el prototipo se conmuta una carga de bajo voltaje por seguridad.
Alimentación	Cargador USB de 5 V para el ESP32; cargador de pared para la laptop y el foco	Suministro del nodo y de los módulos de 5 V (MQ-2, PIR, bobina del relé y buzzer).	Los sensores de 3.3 V (DHT22) y la lógica del relé se toman del regulador del ESP32; los de 5 V, del riel de 5 V de la protoboard.

Categoría	Componente seleccionado	Función técnica en el prototipo	Justificación técnica
Interfaz de familiares	Celular con Telegram y navegador web	Recepción de alertas, chat y uso del panel.	No requiere instalar una aplicación propia; el panel funciona en cualquier navegador.

Componentes de Software y Servicios

- **Plataforma del servidor y lógica central:**
  - Sistema operativo: Ubuntu Server en una máquina virtual, con Docker Compose y reinicio automático de los servicios.
  - Motor de voz local: Vosk (modelo en español, modo gramática con frases señuelo) para el reconocimiento y Piper (voz es_MX-claude-high) para la síntesis. Funciona sin depender de la nube y protege la privacidad.
  - Orquestador: Python con asyncio y base de datos SQLite, en lugar de Home Assistant, para tener control total de las reglas, la cola de alertas y las pruebas automáticas.
  - Panel web: FastAPI (API y WebSocket) y Flutter web, publicado con Tailscale Funnel.
- **Satélite de voz:**
  - Cliente en Python como servicio de usuario en la laptop: captura con PipeWire, envío del audio por WebSocket, Piper para las respuestas y mpv con yt-dlp para los videos.
- **Firmware del ESP32:**
  - C++ con Arduino Core y ESP32MQTTClient (cliente MQTT de ESP-IDF con QoS 1): lectura de sensores, control de pines GPIO, alarma de gas con histéresis y memoria no volátil (NVS) para conservar el estado.
- **Mensajería interna:**
  - Mosquitto MQTT Broker en el servidor como bus de eventos local, con usuario y ACL por componente.
- **Pasarela externa de alertas:**
  - API de bots de Telegram: mensajes a los familiares ante eventos de alerta («Necesito ayuda», «Fuga de gas detectada», inactividad), botones para abrir el panel o marcar la alerta como atendida, y chat de dos vías. Twilio (SMS y llamadas) queda como trabajo futuro.

Cambios respecto a la versión anterior

Elemento	Versión anterior	Implementado	Motivo
Unidad central	Raspberry Pi 4/5 o mini PC como gateway	Máquina virtual en el servidor de la casa, con Docker	Más potencia para la voz y respaldos de la máquina virtual.
Captura de voz	ReSpeaker Mic Array en el gateway	Micrófono inalámbrico en un satélite (laptop) en la estancia	Disponibilidad y funcionamiento en Linux sin controladores.
Activación	Palabra de activación (wake-word)	Tecla o botón del panel mantenido para hablar	Prioridad a la confiabilidad; la palabra de activación queda como trabajo futuro.
Lenguaje natural	Frases libres («hace frío»)	Conjunto cerrado de frases con señuelos	Menos órdenes falsas con frases ajenas y respuesta inmediata.

Elemento	Versión anterior	Implementado	Motivo
Orquestador	Home Assistant o Python	Python (asyncio) con SQLite	Control de las reglas, la cola de alertas y las pruebas.
Enlace externo	Telegram, SMS y llamadas	Telegram y panel web con HTTPS	Telegram cubre las alertas sin costo; SMS y llamadas, a futuro.
Riesgo de gas	Solo detección	Detección con alarma local (buzzer)	La alarma suena aunque falle la red.
Nuevo	—	Panel web, chat de dos vías, recordatorios hablados, consultas y video	Funciones definidas en la Fase 4.